Secure Web Apex

Application, Container Platform, Kubernetes, and CI/CD Delivery Review

Solution	Secure Web Apex
Repository baseline	main at commit a05a724 (12 July 2026)
Deployment target	Google Artifact Registry and Google Kubernetes Engine (asia-south1)
Review method	Static inspection of source, deployment manifests, Git history, and GitHub Actions workflow
Prepared for	Organization-ready technical project documentation

Prepared by Shiba Prasad Praharaj | 12 July 2026

Report position: The delivery automation is a meaningful improvement over the previous report. This revision distinguishes configuration verified in the repository from live-environment results that could not be established by static review.

Professional Development Approach

A candidate statement for interview and portfolio discussions.

How I Developed Secure Web Apex

Secure Web Apex is a hands-on learning project that I developed by bringing together frontend design, PHP database connectivity, containerization, Kubernetes resources, and security-focused thinking. I used ChatGPT and Gemini as learning and productivity assistants to understand concepts, compare approaches, troubleshoot issues, and improve documentation. Android Studio was also part of my development and learning workflow.

I remained responsible for understanding how the components fit together, reviewing suggestions, adapting code to the project requirements, testing flows, and documenting both the strengths and limitations of the implementation. The result is not simply a collection of code: it is a project I can explain, demonstrate, and improve.

What This Project Demonstrates

- Self-directed learning: I moved from a browser-based prototype to PHP, database adapters, Docker, and Kubernetes concepts.
- Practical problem solving: I used AI tools to accelerate research and debugging, then applied the learning to real project components.
- Responsible engineering mindset: I identified prototype limitations such as client-side authorization, plain-text passwords, and example credentials, and documented the production hardening needed.
- Communication skills: I can explain the role of each file, the user workflow, the deployment design, and the trade-offs behind the current implementation.

1. Executive Summary

Secure Web Apex is a role-aware portal project with two distinct operating modes: a browser-based demonstration that stores data in browser storage, and a PHP path intended to use an interchangeable database adapter. The current repository now adds a delivery pipeline that builds an Apache/PHP container, publishes it to Google Artifact Registry, and deploys an image identified by the Git commit SHA to Google Kubernetes Engine (GKE).

This report supersedes the 10 July 2026 project report. It updates the Docker build context, Kubernetes deployment model, repository structure, and the newly restored GitHub Actions workflow. It also corrects prior documentation that treated persistent-volume manifests as current repository artifacts; they are no longer present in the tracked tree.

1.1. Current position

Area	Repository-confirmed status	Business interpretation
Application	Browser demo and PHP adapter source are present.	Useful as a learning/prototype solution; access controls are not yet a production security boundary.
Container	PHP 8.1 Apache image installs PDO MySQL/PostgreSQL drivers and the MongoDB extension.	Container packaging is configured; runtime dependencies still need validation.
Kubernetes	App Deployment, public LoadBalancer Service, and HPA are declared.	Application platform configuration is present; database provisioning is incomplete in the tracked manifests.
CI/CD	A main-branch GitHub Actions workflow builds, publishes, deploys, and waits for rollout status.	There is now a traceable automated path from source change to GKE deployment.

Important qualification: This is a repository-based technical assessment. No Docker build, GitHub Actions run, Artifact Registry inspection, GKE deployment, or production data path was executed as part of this report.

1.2. Software and Platform Versions

Component	Version / status observed	Purpose
PHP Apache base image	php:8.1-apache (pinned in Dockerfile)	Serves the PHP web application.
PHP database support	PDO, pdo_mysql, pdo_pgsql; MongoDB extension installed	Connects the adapter layer to relational or MongoDB back ends.
Kubernetes APIs	apps/v1; autoscaling/v2; v1	Deployment, Service, HPA, PV, and PVC resources.
MongoDB container	mongo:latest (not version-pinned)	Kubernetes database workload; port 27017.
Browser APIs	Modern browser required	localStorage/sessionStorage, camera access, and File System Access API (local backup).
MySQL / AWS RDS	Version not specified	Schema and PHP PDO adapter support database-backed practical deployment.
PostgreSQL	Version not specified	PDO adapter is present; no database manifest is included.

What changed since the prior report

- A GitHub Actions workflow now authenticates to Google Cloud through GitHub OIDC / Workload Identity Federation, rather than storing a static cloud key in the repository.
- The build now deliberately uses the repository root as Docker context and secure-wep-apex/Dockerfile as the Dockerfile, while copying only the application subfolder into the image.
- The deployment manifest receives the project ID and immutable Git SHA image tag during the workflow, then the workflow checks GKE rollout completion for deployment/secure-web-apex.
- The app Deployment name, HPA target, Artifact Registry location, resource sizing, and replica ceiling have changed. Persistent-volume manifests have been removed from the tracked repository, although db.yml still references mongodb-pvc.

2. Scope, Evidence, Functional Use Cases and Architecture

Review scope included the tracked source tree, Git history, Dockerfile, Kubernetes manifests, PHP adapter layer, browser client, README, and the GitHub Actions workflow. The repository is organized with automation at the root and the web application in the secure-wep-apex subfolder.

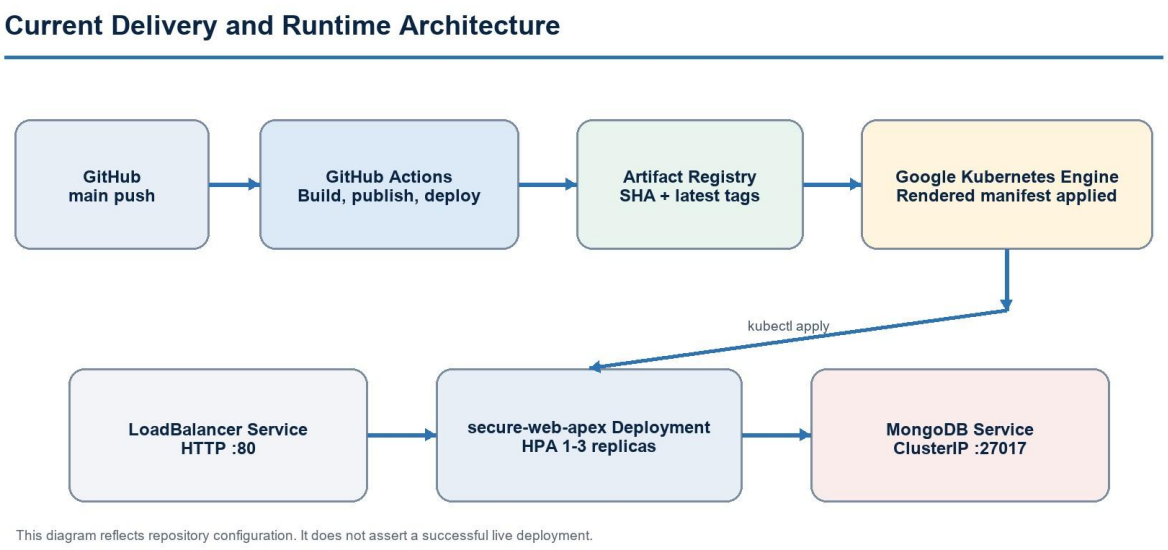


Figure 1. Delivery and runtime architecture reflected in the current repository configuration.

2.1. Evidence baseline

Evidence	Finding
Git branch and commit	main at a05a724, titled 'Modify Docker workflow for GKE deployment', dated 12 July 2026.
Workflow	.github/workflows/docker-image.yml is tracked and runs on a push to main.

Application package	The secure-wep-apex subfolder contains browser assets, PHP pages, database SQL, Dockerfile, k8s.yaml, and db.yml.
Evidence	**Finding**
Change record	Git history shows Dockerfile and k8s.yaml updates plus addition/revision of the GKE deployment workflow after the prior report date.
Static review boundary	No test reports, deployment evidence, code-quality gates, container scan output, or successful release record was found in the inspected files.

Repository evidence: git log, git diff since 10 July 2026, .github/workflows/docker-image.yml, secure-wep-apex/Dockerfile, secure-wepapex/k8s.yaml, and current tracked-file inventory.

2.2. Functional Use Cases

Actor	Use case	Result
Employee	Register and sign in; view own dashboard	Uses employee portal and own records.
Manager	Sign in with additional verification; manage departmental users	Views and manages employee data within the configured department.
HR	Access personnel-oriented records	Views manager and employee records; can edit where UI logic permits.
CEO	Use executive dashboard and backup control	Can access broader data and initiate backup workflow after extra verification.
Administrator / operator	Select database type through environment variables	Runs the same PHP application with memory, MySQL/MariaDB/RDS, PostgreSQL, or MongoDB adapter selection.
Platform operator	Deploy to Kubernetes	Exposes app through LoadBalancer and scales application replicas based on CPU utilization.

2.3. Application and Data Layer

The front-end demonstration is a static HTML/CSS/JavaScript portal. It supports employee, manager, HR, and CEO personas, registration and sign-in flows, directory and profile interactions, attendance logging, internal messaging, and backup simulation. These flows are implemented in assets/js/app.js with localStorage for data and sessionStorage for the current-user identifier.

The PHP path provides a small form submission and dashboard experience. php/config.php reads DB_TYPE and related connection settings from environment variables; DatabaseFactory selects a MariaDB/MySQL/RDS, PostgreSQL, MongoDB, or memory adapter. php/submit.php validates basic required fields and email syntax before calling insertUser; php/dashboard.php renders the retrieved user list.

2.4. Application interface evidence

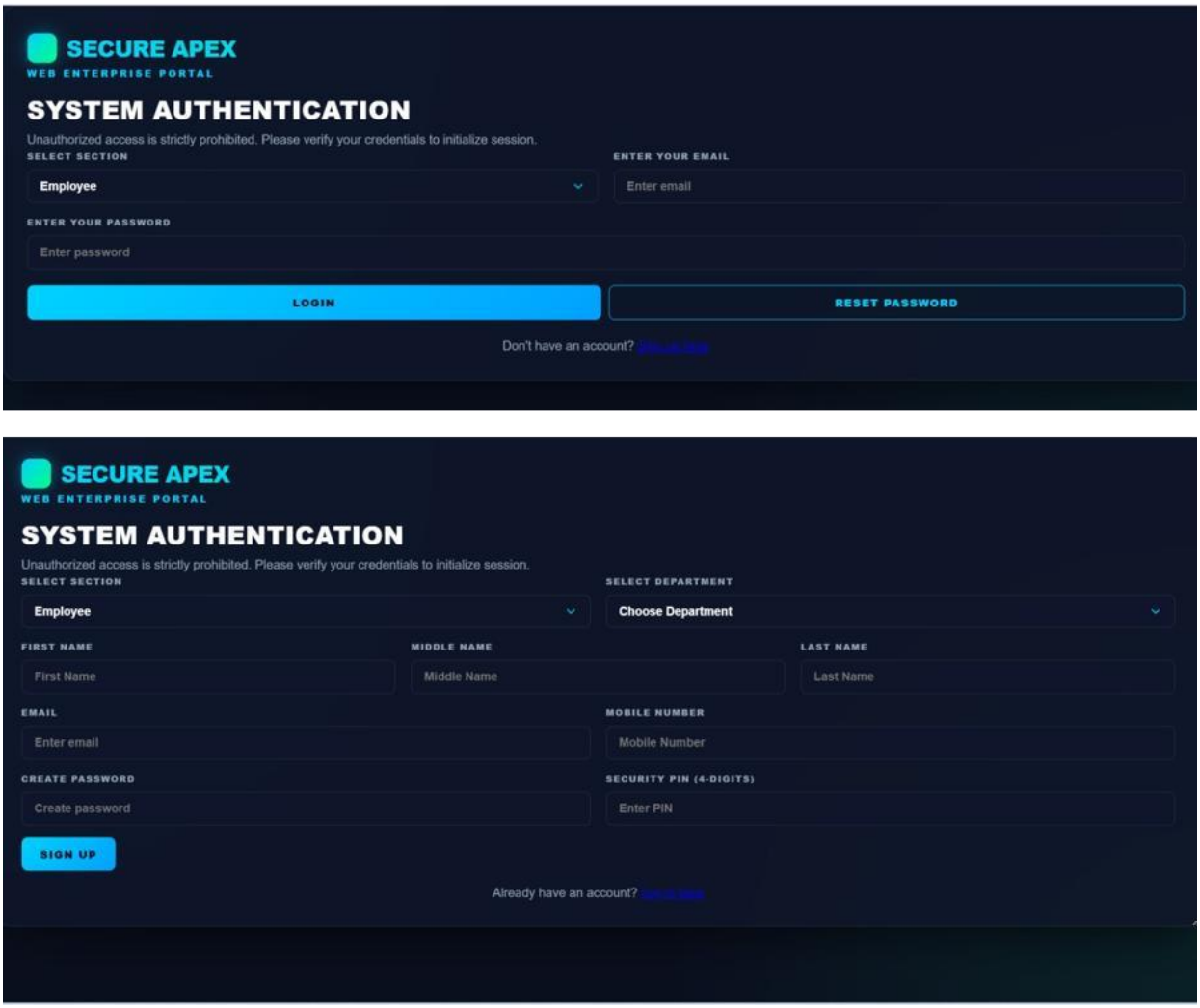


Figure 2. Supplied Secure Apex portal capture showing the system-authentication and self-registration interface.

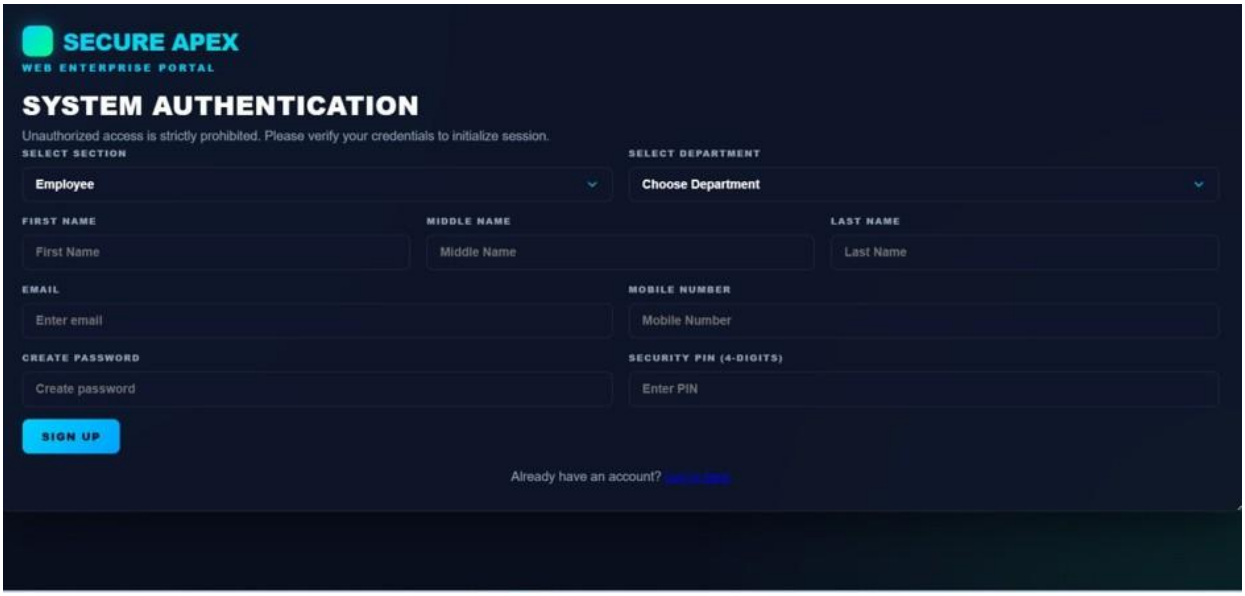


Figure 3. Supplied Secure Apex self-registration screen, including department, identity, contact, password, and security-PIN fields.

Component	Current behavior	Use / limitation
Browser demo	Client logic stores identity, attendance, messages, and backup state in browser storage.	Convenient for demonstrations; client-side state must not be treated as authoritative identity, authorization, or protected data storage.
PHP data interface	DBAdapter exposes connect, insertUser, getAllUsers, and close; DatabaseFactory selects the backend based on DB_TYPE.	An extensibility pattern is present; full production data access, authorization, and error handling are not implemented.
SQL schema	database.sql defines users, messages, and notifications for MySQL / RDS practical work.	The schema carries sensitive fields and a clear-text password column; it must be redesigned before real personal data is processed.
MongoDB path	k8s.yaml selects MongoDB and db.yml exposes mongodb-service internally.	The supplied source lacks composer.json and the mongodb/mongodb library that supplies MongoDB\Client; the MongoDB PHP path therefore needs dependency completion and runtime testing.
Memory path	MemoryAdapter uses sqlite::memory:.	The shown Dockerfile does not install pdo_sqlite, so the default memory path needs a runtime dependency correction before it can be relied upon in the container.

Design interpretation: The adapter pattern makes database selection explicit, but it is not a substitute for verified database drivers, schema alignment, migration management, secret delivery, or server-side user authorization.

Repository evidence: secure-wep-apex/index.html, dashboard.html, assets/js/app.js, php/config.php, php/db.php, php/submit.php, php/dashboard.php, and database/database.sql.

3. Container Packaging

The container definition begins with php:8.1-apache, installs libpq development libraries, PDO drivers for MySQL and PostgreSQL, the PECL MongoDB extension, and selected utility packages. It builds from the repository root but copies only secure-wep-apex/ into /usr/src/app, then copies those prepared application files to /var/www/html. Apache is exposed on port 80 and started with apache2-foreground.

Previous assumption / configuration	Current configuration	Operational effect
COPY . . in the build stage	COPY secure-wep-apex/ .	The image excludes root-level workflow material and copies the intended application folder when the workflow builds with repository-root context.
Dockerfile path was not coordinated with a root-context pipeline	Workflow explicitly runs docker build -f secure-wep-apex/Dockerfile	Build context and Dockerfile location are aligned for GitHub Actions.
Image startup syntax was previously unstable in history	CMD ["apache2-foreground"]	The current form is valid exec-form Dockerfile syntax.
A builder / runner label suggests a multi-stage image	Runner is derived FROM builder and copies its full prepared app output.	The implementation is operationally structured as two named stages, but it does not reduce runtime package footprint because the runner inherits the builder image.

4. Dockerfile update

secure-wep-apex > Dockerfile > ...

```
FROM php:8.1-apache AS builder

RUN apt-get update \
    && apt-get install -y --no-install-recommends \
        libpq-dev \
        libssl-dev \
        zip \
        unzip \
        git \
    && docker-php-ext-install pdo pdo_mysql pdo_pgsql \
    && pecl install mongodb \
    && docker-php-ext-enable mongodb \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /usr/src/app

# 👇 CHANGE: Copy ONLY the contents of your app folder into the workdir
COPY secure-wep-apex/ .

# Optional Composer install if composer.json exists
RUN if [ -f composer.json ]; then \
        curl -sS https://getcomposer.org/installer | php && php composer.phar install --no-dev --optimize-autoloader; \
    fi

RUN chown -R www-data:www-data /usr/src/app \
    && find /usr/src/app -type d -exec chmod 755 {} \; \
    && find /usr/src/app -type f -exec chmod 644 {} \;

# Stage 2: runtime stage uses the prepared builder image and copies final app files
FROM builder AS runner
WORKDIR /var/www/html
COPY --from=builder /usr/src/app /var/www/html

ENV PORT=80
EXPOSE 80
CMD ["apache2-foreground"]
```

4.1. Container readiness considerations

- Pin the base image and installed package versions or digests, and use a maintained PHP release before production use.
- Install the required Composer MongoDB client library if the MongoDB adapter is retained; validate the image with DB_TYPE=mongodb against a controlled test database.
- Add pdo_sqlite if the documented memory mode is intended to run inside this image.
- Add a non-root / least-privilege runtime configuration, read-only filesystem where feasible, and a health endpoint that Kubernetes can probe.
- Add a reproducible test stage and image scan before publication; the current workflow builds and deploys without source tests or security scanning.

Repository evidence: secure-wep-apex/Dockerfile and .dockerignore; comparison against the pre-update Dockerfile in Git history.

5. Kubernetes Deployment Design

k8s.yaml defines the application Deployment, a type LoadBalancer Service, and an autoscaling/v2 HorizontalPodAutoscaler. The workflow renders the application image reference by replacing ${PROJECT_ID} and ${IMAGE_TAG} before applying the manifest. The deployed image therefore uses the immutable Git SHA tag generated in the same CI/CD run.

Resource	Current declaration	Implementation note
Deployment	secure-web-apex; selector and pod label app: secureapex.	The deployment name now matches the workflow rollout command and HPA target.
Container image	asia-south1-docker.pkg.dev/${PROJECT_ID}/gcp-cicdrepo/my_container:${IMAGE_TAG}	Placeholders are rendered by sed in the workflow; manifest application should fail safely if substitutions are not made.
Resources	Requests: 250m CPU / 512Mi; limits: 2 CPU / 2Gi.	HPA CPU utilization is calculated from requested CPU. Capacity planning should validate these numbers under expected load.
Service	secure-apex-service, type LoadBalancer, port 80.	This exposes HTTP directly; an HTTPS ingress, certificate management, and web application protections are not declared.
HPA	Minimum 1, maximum 3 replicas; 50% average CPU target.	A metrics pipeline such as Metrics Server is required for the HPA to make scaling decisions.
Database manifest	db.yml declares MongoDB and mongodb-service with a PVC mount.	db.yml is not applied by the workflow, references mongodb-pvc, and current tracked files do not include a PVC/PV definition. Database deployment is therefore a prerequisite outside the current app release manifest.

Deployment constraint: The absence of a replicas field is compatible with the HPA model; Kubernetes defaults the Deployment to one replica and the HPA governs the 1-3 range when metrics are available. It does not make the app highly available by itself; PodDisruptionBudgets, probes, zones, and rollout safeguards remain relevant.

5.1. Configuration to add before a production rollout

- Kubernetes Secrets or a managed secret integration for DB_USER and DB_PASS, with credential rotation. Do not retain the example password in manifests.
- A durable database design: provision and apply storage resources, pin the MongoDB image, define backups, or use a managed database service.
- readinessProbe and livenessProbe, securityContext, resource monitoring, NetworkPolicies, namespace boundaries, and a narrowly scoped Kubernetes service account.
- TLS termination through an ingress or load balancer configuration, plus an explicit firewall and DNS strategy.

Repository evidence: secure-wep-apex/k8s.yaml and secure-wep-apex/db.yml; current repository tree shows pv.yml and pvc.yml are not tracked.

```yaml
# # ========================================
# APPLICATION DEPLOYMENT SECTION
# ========================================
apiVersion: apps/v1
kind: Deployment
metadata:
  name: secure-web-apex
  labels:
    app: secure-apex
spec:
  # Removed hardcoded 'replicas' to let HPA handle scaling seamlessly
  selector:
    matchLabels:
      app: secure-apex
  template:
    metadata:
      labels:
        app: secure-apex
    spec:
      containers:
      - name: secure-apex-portal
        # Kept the exact placeholders for dynamic replacement in the pipeline
        image: asia-south1-docker.pkg.dev/${PROJECT_ID}/gcp-cicd-repo/my_container:${IMAGE_TAG}
        imagePullPolicy: Always
        env:
        - name: DB_TYPE
          value: "mongodb"
        - name: DB_HOST
          value: "mongodb-service"
        - name: DB_PORT
          value: "27017"
        - name: DB_USER
          value: "admin"
        - name: DB_PASS
          value: "password"
        - name: DB_NAME
          value: "secure_apex"
```

```
        ports:
        - containerPort: 80
        resources:
          requests:
            cpu: "250m"       # Lowered to 0.25 core so pods schedule reliably
            memory: "512Mi"   # Lowered to 512MB initial request
          limits:
            cpu: "2"          # Kept high to allow bursting
            memory: "2Gi"     # Kept high to handle spikes
---
# ========================================
# APPLICATION SERVICE SECTION
# ========================================
apiVersion: v1
kind: Service
metadata:
  name: secure-apex-service
spec:
  selector:
    app: secure-apex
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
  type: LoadBalancer
---
# ========================================
# HORIZONTAL POD AUTOSCALER (HPA) SECTION
# ========================================
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
```

```
kind: HorizontalPodAutoscaler
metadata:
  name: secure-apex-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: secure-web-apex
  minReplicas: 1
  maxReplicas: 3 # Increased slightly to allow scaling beyond minimum if CPU triggers
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

6. GitHub Actions CI/CD Workflow

The workflow is named secure-webapex.pipeline and triggers on pushes to main. It uses GitHub's id-token permission and google-github-actions/auth@v2 with a Workload Identity Federation provider and serviceaccount email supplied as GitHub secrets. This is a positive cloud-authentication design because it supports short-lived access tokens instead of embedding a static Google Cloud key in the workflow.

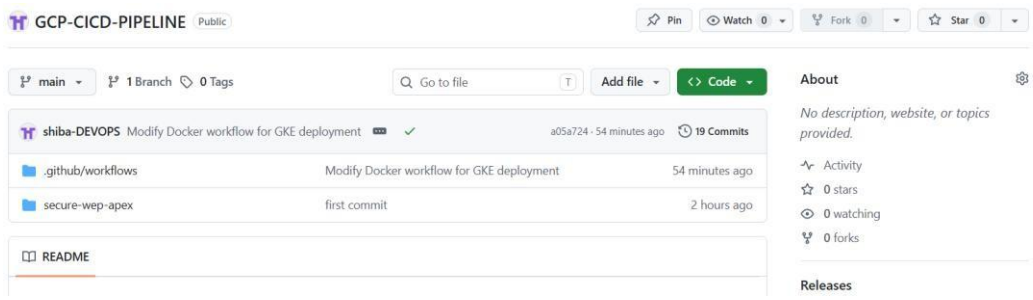
Deployment evidence captured after CI/CD implementation

The following supplied output captures document the implemented workflow and its GKE result. They are presented as project evidence separate from the repository review: the images show a completed GitHub Actions job, an available GKE workload, and a running deployment revision with a LoadBalancer service.

Code Blame 63 lines (52 loc) · 2.35 KB Raw

```yaml
name: secure-webapex.pipeline

on:
  push:
    branches: [main]

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      id-token: write

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - id: auth
        name: Authentication to Google Cloud
        uses: google-github-actions/auth@v2
        with:
          token_format: access_token
          workload_identity_provider: ${{ secrets.WIF_PROVIDER }}
          service_account: ${{ secrets.GCP_SA_EMAIL }}

      - name: Docker Auth
        run: |
          echo "${{ steps.auth.outputs.access_token }}" | docker login -u oauth2accesstoken --password-stdin https://asia-south1-docker.pkg.dev

      - name: Build and Push container
```

```yaml
      - name: Build and Push container
        run: |
          # Build using the subfolder path while keeping context at root '.'
          docker build -f secure-wep-apex/Dockerfile \
            -t asia-south1-docker.pkg.dev/${{ secrets.GCP_PROJECT_ID }}/gcp-cicd-repo/my_container:${{ github.sha }} \
            -t asia-south1-docker.pkg.dev/${{ secrets.GCP_PROJECT_ID }}/gcp-cicd-repo/my_container:latest .
          docker push asia-south1-docker.pkg.dev/${{ secrets.GCP_PROJECT_ID }}/gcp-cicd-repo/my_container:${{ github.sha }}
          docker push asia-south1-docker.pkg.dev/${{ secrets.GCP_PROJECT_ID }}/gcp-cicd-repo/my_container:latest

      # CD PIPELINE
      - name: Set up Google Cloud SDK
        uses: google-github-actions/setup-gcloud@v2
        with:
          project_id: ${{ secrets.GCP_PROJECT_ID }}
          # Installs the missing authentication plugin required for kubectl
          install_components: 'gke-gcloud-auth-plugin'

      - name: Get GKE credentials
        run: |
          gcloud container clusters get-credentials secure-web-apex-cluster \
            --location asia-south1 \
            --project ${{ secrets.GCP_PROJECT_ID }}

      - name: Deploy to GKE
        run: |
          # Process the k8s template from the subfolder
          sed -e "s|\${PROJECT_ID}|${{ secrets.GCP_PROJECT_ID }}|g" \
              -e "s|\${IMAGE_TAG}|${{ github.sha }}|g" \
              secure-wep-apex/k8s.yaml > k8s-deploy.yaml

          kubectl apply -f k8s-deploy.yaml

          # Tracks deployment rollout health using the exact deployment name
          kubectl rollout status deployment/secure-web-apex --timeout=120s
```



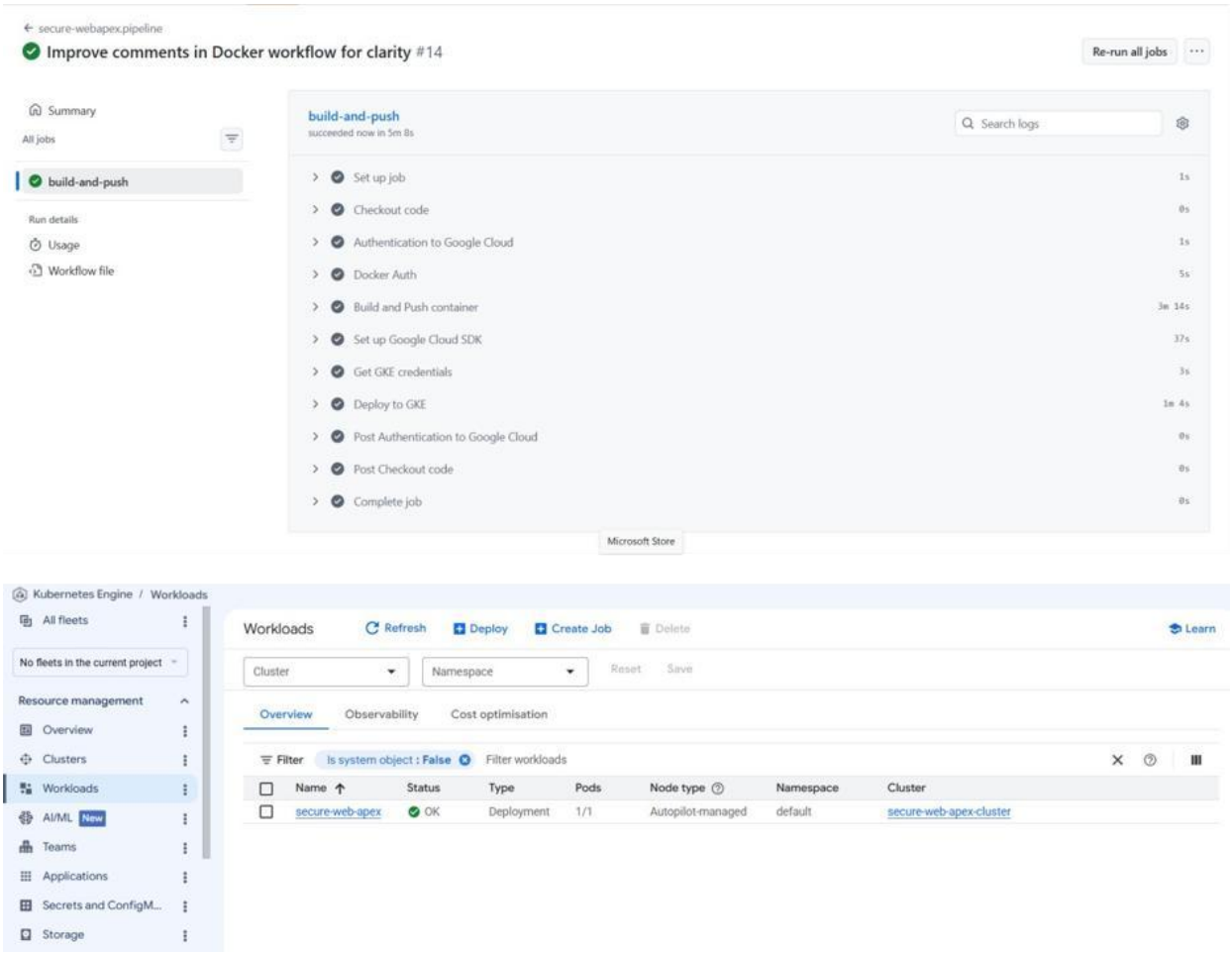


Figure 4. Supplied CI/CD output: GitHub Actions build-and-push job completed successfully, with the related GKE workload view shown below.

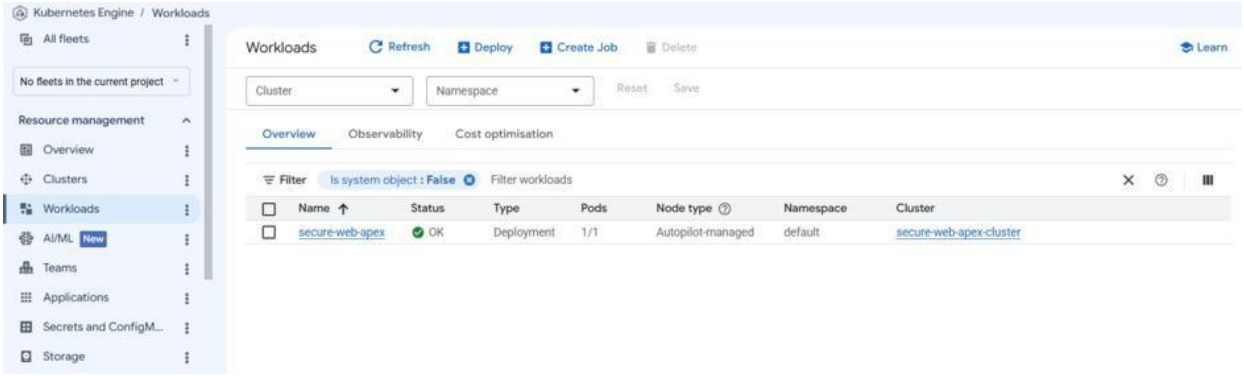


Figure 5. Supplied GKE Workloads output: secure-web-apex is displayed as OK with one ready pod in the default namespace.

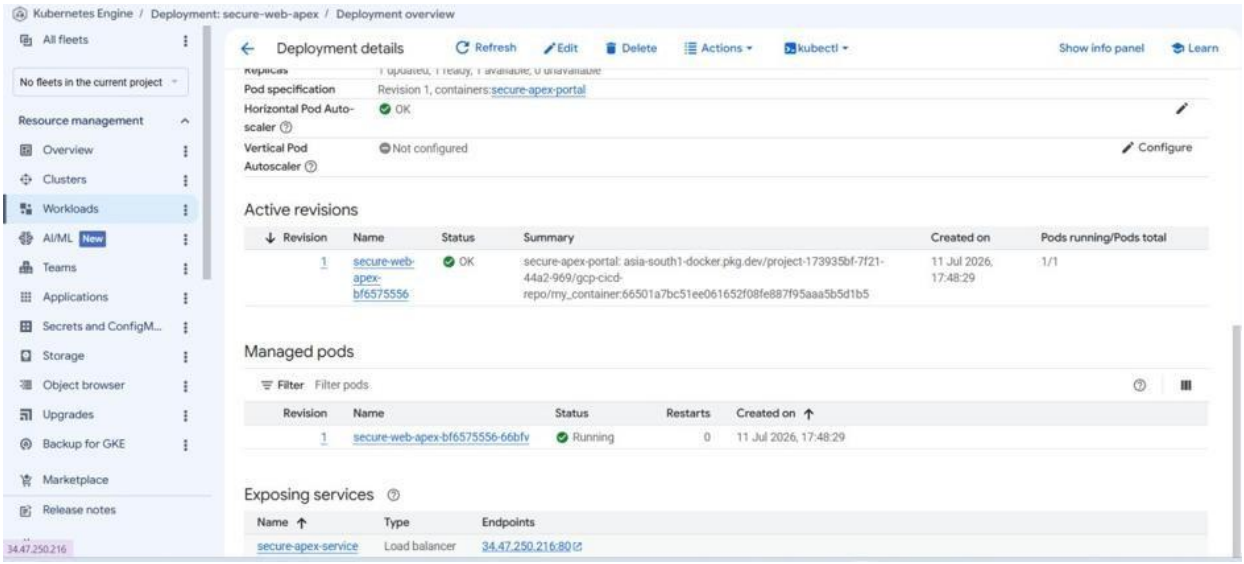


Figure 6. Supplied GKE Deployment output: active revision is OK, managed pod is running, and secure-apex-service is exposed as a LoadBalancer.

Stage	Implemented action	Control / outcome
1. Source	actions/checkout@v4 checks out the main-branch commit.	The build has a defined source revision.
2. Cloud auth	OIDC authentication exchanges the GitHub identity for a Google access token.	Requires correct Workload Identity Provider, service account, trust conditions, and IAM roles.
3. Registry login	Docker logs into asia-south1-docker.pkg.dev using the shortlived token.	Avoids registry password storage; access scope should be restricted to the required repository.
4. Build and publish	Builds with the subfolder Dockerfile and root context; pushes both github.sha and latest tags.	The SHA tag provides the immutable deployment artifact; latest is a convenience tag and should not drive promotion decisions.
5. Cluster auth	setup-gcloud installs the GKE authentication plugin; gcloud retrieves credentials for secure-web-apex-cluster in asiasouth1.	Requires cluster access and correct regional cluster configuration.
6. Deploy	sed replaces manifest placeholders; kubectl apply applies the rendered file; rollout status waits 120 seconds.	Creates a direct CI-to-GKE delivery path and fails the job if the named deployment does not complete rollout in the configured time.

Release-control observations

- The pipeline has no pull-request trigger, automated tests, linting, container vulnerability scan, policy check, manifest validation, or staged environment separation in the tracked workflow.
- The pipeline deploys the commit SHA, which is the correct primary release reference. Retain the SHA through approval, observability, rollback, and release notes rather than relying on latest.
- Use GitHub Environments for production with protected reviewers, concurrency control to avoid overlapping deployments, and an explicit rollback procedure such as kubectl rollout undo deployment/secure-web-apex.
- Restrict the workload identity trust policy to the expected repository, branch, and environment claims; restrict the service account to only Artifact Registry and GKE actions required for this workload.

Repository evidence: .github/workflows/docker-image.yml and the Git commit history from 11-12 July 2026.

Release Change Ledger

The following ledger captures the report-relevant changes between the previous project-report baseline and the current repository state. It is intentionally specific to observable source control changes.

Area	Earlier report / baseline	Current state	Impact
Repository automation	No current CI/CD workflow was documented as a release path.	GitHub Actions workflow builds, publishes, gets GKE credentials, applies the rendered manifest, and checks rollout status.	Automated delivery capability added.
Build context	Application build copied the context without the later root/subfolder coordination.	Workflow uses the root context and Dockerfile copies secure-wep-apex/ only.	Repository layout and container build are aligned.
Registry and tag	Earlier Kubernetes image reference was a fixed project/location path.	Image path uses Asia South Artifact Registry placeholders; workflow deploys a Git SHA tag.	Deployment artifact becomes traceable to source commit.
Deployment target	Deployment name in the earlier manifest did not match the current rollout name.	Deployment is secure-web-apex and workflow checks that same deployment.	Rollout status now targets the declared resource.
Capacity	100m / 128Mi requests, 500m / 256Mi limits; HPA maximum 5.	250m / 512Mi requests, 2 CPU / 2Gi limits; HPA maximum 3.	Scheduling and scaling behavior changed; needs load validation.
Persistence	PV and PVC were reported as project files.	PV/PVC manifests are deleted from the current tracked tree; db.yml still expects mongodb-pvc.	Database storage is not selfcontained in the current release assets.

Documentation correction: The updated report does not list pv.yml or pvc.yml as current project artifacts. Their removal is material because the remaining MongoDB manifest references a PVC that is not defined by the current tracked source.

7. Security and Production Readiness

Secure Web Apex demonstrates several useful foundations: parameterized SQL inserts for the SQL adapters, environment-based configuration selection, internal ClusterIP service exposure for MongoDB, container resource requests and limits, HPA configuration, and workload-identity-based cloud authentication in the delivery workflow. These measures are helpful but are not sufficient for internet-facing or personal-data processing use.

Priority	Observed condition	Required treatment
Critical	Passwords are stored and displayed in clear text in the browser demonstration, SQL schema, PHP path, and dashboard.	Use password_hash / password_verify, remove password fields from responses, build an authenticated reset flow, and migrate or delete unsafe test records.

Critical	Database credentials are literal values in k8s.yaml and db.yml.	Use Kubernetes Secrets or a managed secret system; remove sample credentials from deployable manifests; rotate immediately if ever used outside a demo.
High	Browser localStorage/sessionStorage controls identity and role presentation.	Move authentication, sessions, authorization checks, audit events, and sensitive records to the server; treat browser UI roles as presentation only.
High	MongoDB deployment uses mongo:latest and references a missing PVC definition; PHP Mongo client dependency is not shown.	Pin an approved image/digest, establish durable encrypted storage and backups, complete dependencies, and run integration tests.
High	No probes, TLS ingress, network policies, pod security context, or image security gate are declared.	Add platform hardening, HTTPS, network segmentation, runtime controls, vulnerability scanning, and admission/policy checks.
Medium	CI/CD deploys directly from main without PR quality gates or protected environment approvals.	Add CI checks, release approval, immutable promotion, observability thresholds, and controlled rollback.

Recommended readiness sequence

- First, remove clear-text credential handling and establish a server-side authentication and authorization model.
- Second, make the database path deployable and durable: secret delivery, pinned dependencies, schema/migration strategy, backups, storage, and integration testing.
- Third, add container and Kubernetes hardening, HTTPS, monitoring, alerting, log retention, and incidentready rollback controls.
- Finally, introduce PR checks and environment-based deployment promotion so the current main-push workflow becomes a controlled release process.

Repository evidence: assets/js/app.js, php/db.php, php/dashboard.php, database/database.sql, Dockerfile, k8s.yaml, db.yml, and docker-image.yml.

8. Operating Model and Deployment Prerequisites

The current workflow can be used as a starting point for an organization-controlled delivery model once the following prerequisites are fulfilled. These conditions are requirements inferred from the tracked configuration; their existence was not verified as part of the static review.

Domain	Required condition	Owner-facing outcome
GitHub	WIF_PROVIDER, GCP_SA_EMAIL, and GCP_PROJECT_ID secrets are configured; main branch is protected; production environment is controlled.	The workflow can obtain short-lived cloud credentials without a committed key.
Google Cloud	Artifact Registry repository gcp-cicd-repo exists in asia-south1; the WIF service account has scoped permissions for the repository and target GKE cluster.	Build publication and cluster access can proceed predictably.

GKE	Regional cluster secure-web-apex-cluster exists in asia-south1; kubectl authentication works; metrics infrastructure is available for HPA.	The workflow can render, apply, and observe the declared application Deployment.
Database	MongoDB service and durable storage are deliberately provisioned, or application configuration is changed to a validated managed database backend.	The app's selected DB_TYPE has a reachable and supported runtime path.
Security	Secrets, ingress/TLS, network policy, probes, pod security, logging, monitoring, and rollback control are approved.	The release meets a minimum operational ownership model.

Suggested deployment flow

- A reviewed change merges to main after automated tests, container checks, and manifest validation succeed.
- The workflow authenticates through workload identity, builds the image from the exact commit, and publishes the Git SHA tag to Artifact Registry.
- The workflow renders k8s.yaml with the target project ID and Git SHA, applies it to the approved GKE environment, and waits for rollout status.
- Post-deployment monitors verify service health. A failed release is stopped and reverted to a known SHA using the deployment rollout history.

Operational guardrail: The present rollout check verifies only Kubernetes Deployment rollout status. It should be supplemented by readiness probes, application smoke tests, service-level monitoring, and a defined rollback owner before it is used for production releases.

9. Representative Code Walkthroughs

9.1. Browser authentication form- index.html

This form collects a selected role, email address, and password. JavaScript attaches the event handler and routes the user only after the browser-demo checks succeed. In a production application, the same data must be sent over HTTPS to a server-side authentication service.

```
        <form id="loginForm" class="entry-form" autocomplete="off">
            <div class="field-grid">
                <div>
                    <label for="loginRole">Select Section</label>
                    <select id="loginRole" name="corporate-role-selection" required>
                        <option value="employee">Employee</option>
                        <option value="manager">Manager</option>
                        <option value="hr">HR</option>
                        <option value="ceo">CEO</option>
                    </select>
                </div>
                <div>
                    <label for="loginEmail">Enter Your Email</label>
                    <input type="email" id="loginEmail" name="auth-identity-field" placeholder="Enter email"
required                            autocomplete="one-time-code" readonly
onfocus="this.removeAttribute('readonly');">                </div>
            </div>
            <div class="field-grid" style="margin-bottom: 20px;">
                <div>
                    <label for="loginPassword">Enter Your Password</label>
                    <input type="password" id="loginPassword" name="auth-security-key" placeholder="Enter password"
required                            autocomplete="one-time-code" readonly onfocus="this.removeAttribute('readonly');">
</div>
            </div>
            <div style="display: flex; gap: 10px;">
```

9.2. Client session and storage keys- assets/js/app.js

These constants isolate browser keys for users, the active session, attendance, messages, and backup state. The helper functions read and write localStorage while sessionStorage keeps the active-user identifier for the browser session. This is convenient for a demo, but must not be used as the final authority for identity or permissions.

```
const storageKey = "secure-web-apex_users"; const
sessionKey = "secure-web-apex_current_user_id"; const
messagesKey = "secure-web-apex_messages"; const
attendanceKey = "secure-web-apex_attendance_log"; const
backupStatusKey = "secure-web-apex_backup_active";


// Data Migration Bridge
if (localStorage.getItem("aws_rds_practical_users") && !localStorage.getItem(storageKey))
{    localStorage.setItem(storageKey, localStorage.getItem("aws_rds_practical_users")); }


// --- CORE UTILITIES --- function getStoredUsers() { return
JSON.parse(localStorage.getItem(storageKey) || "[]"); } function saveUsers(users) {
localStorage.setItem(storageKey, JSON.stringify(users)); }


// Role Migration Bridge (function
migrateRoles() {    const users =
getStoredUsers();    let migrated
= false;    users.forEach(u => {
```

```
        if (u.role === 'head') {
u.role = 'ceo';
migrated = true;
        }
});
    if (migrated) saveUsers(users);
})();
function getCurrentUserId() { return sessionStorage.getItem(sessionKey); }
function setCurrentUserId(userId) { sessionStorage.setItem(sessionKey, userId); }
function clearCurrentUserId() { sessionStorage.removeItem(sessionKey); }


function showNotification(message, type = 'info') {
    if (localStorage.getItem(backupStatusKey) === "true") return;
const container = document.getElementById('notificationContainer');
```

9.3. Database selection- php/config.php

The application reads configuration from environment variables, then DatabaseFactory chooses a matching adapter. This design avoids changing application calls when the selected database engine changes. Credentials should be supplied by secrets or a secure deployment environment, never committed in source files.

```
<?php
require_once 'db.php';


// Dynamic Environment Loading
define('DB_TYPE', getenv('DB_TYPE') ?: 'memory'); // Default to stateless memory
mode define('DB_HOST', getenv('DB_HOST') ?: 'localhost'); define('DB_NAME',
getenv('DB_NAME') ?: 'secure_apex'); define('DB_USER', getenv('DB_USER') ?: 'root');
define('DB_PASS', getenv('DB_PASS') ?: ''); define('DB_PORT', getenv('DB_PORT') ?:
'3306');


// Statelessness: Ensure no local data storage ini_set('session.save_handler',
'files');
ini_set('session.save_path', '/tmp'); // Use RAM-backed tmp directory


class DatabaseFactory {
    public static function create(): DBAdapter
{       switch (strtolower(DB_TYPE)) {
case 'mariadb':             case 'mysql':
case 'rds':
                return new MariaDBAdapter(DB_HOST, DB_NAME, DB_USER, DB_PASS,
DB_PORT);           case 'postgres':            case 'postgresql':
                return new PostgreSQLAdapter(DB_HOST, DB_NAME, DB_USER, DB_PASS, DB_PORT);
case 'mongodb':
                return new MongoDBAdapter(DB_HOST, DB_NAME, DB_USER, DB_PASS,
DB_PORT);            case 'memory':             default:                return new
MemoryAdapter();
        }
}
}


function getDB(): DBAdapter {
static $db = null;
```

9.4. Database schema- database/database.sql

The SQL script creates the primary users table plus messaging and notification records. The email field is unique and the foreign-key relationships automatically remove dependent messages and notifications when a user is deleted. Sensitive columns such as password, face data, salary, and blood group require additional protection before real use.

```
CREATE DATABASE IF NOT EXISTS `secure-web-apex`;
USE `secure-web-apex`;


CREATE TABLE IF NOT EXISTS users (
id INT AUTO_INCREMENT PRIMARY KEY,
    role ENUM('employee', 'manager') NOT NULL DEFAULT 'employee',
    department ENUM('IT', 'HR', 'SALES', 'SOFTWARE DEVELOPMENT', 'SOFTWARE TESTING', 'MARKETING') NOT
NULL,     name VARCHAR(100) NOT NULL,     email VARCHAR(150) NOT NULL UNIQUE,     password VARCHAR(100) NOT
NULL,     job_title VARCHAR(120) DEFAULT NULL,     nationality VARCHAR(100) DEFAULT NULL,     salary
DECIMAL(12,2) DEFAULT NULL,     age INT DEFAULT NULL,     blood_group VARCHAR(10) DEFAULT NULL,
mobile_number VARCHAR(20) DEFAULT NULL,     face_data LONGTEXT DEFAULT NULL,     face_image LONGTEXT
DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);


CREATE TABLE IF NOT EXISTS messages (
id INT AUTO_INCREMENT PRIMARY KEY,
sender_id INT NOT NULL,
receiver_id INT NOT NULL,
    subject VARCHAR(255) DEFAULT 'No Subject',
body TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (sender_id) REFERENCES users(id) ON DELETE CASCADE,
FOREIGN KEY (receiver_id) REFERENCES users(id) ON DELETE CASCADE );


CREATE TABLE IF NOT EXISTS notifications
(    id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL,    message TEXT NOT
NULL,    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

10. Conclusion

Secure Web Apex now shows a clearer end-to-end delivery story: a role-aware portal prototype, adapteroriented PHP data layer, container packaging, Kubernetes deployment artifacts, and a GitHub Actions workflow that publishes an immutable image and deploys it to GKE. The change from a static image reference to a rendered SHA-tagged Artifact Registry image is especially valuable because it improves release traceability.

The project remains best characterized as a learning and prototype implementation rather than a productionready corporate portal. The highest-priority next work is not more delivery automation; it is completing the server-side security model, making the database path valid and durable, removing literal credentials and clear-text passwords, and adding quality/security gates around the release path.

Management takeaway: The repository now provides a credible CI/CD and Kubernetes learning demonstration. Production adoption should be conditional on completion of the critical and high-priority security, persistence, dependency, and operational controls recorded in Section 8.

10.1. Appendix A. Current tracked delivery artifacts

Path	Purpose
.github/workflows/docker-image.yml	GitHub Actions build, Artifact Registry publication, GKE credential retrieval, manifest rendering, deployment, and rollout status check.
secure-wep-apex/Dockerfile	PHP 8.1 Apache container definition with database extension installation and application copy.
secure-wep-apex/k8s.yaml	Application Deployment, public Service, and HPA with workflow-rendered image reference.
secure-wep-apex/db.yml	Separate MongoDB Deployment and ClusterIP Service; references mongodb-pvc.
secure-wep-apex/php/	PHP configuration, adapter layer, form submit handler, and dashboard.
secure-wep-apex/assets/ and HTML	Browser demonstration UI, styling, and client-side workflow logic.
secure-wepapex/database/database.sql	MySQL / RDS-oriented users, messages, and notifications schema.

Repository evidence: Current git ls-files inventory at commit a05a724.